OlliteRT 本地大模型 Spring AI 对接文档

基于红米 K50 手机端 OlliteRT 服务 · OpenAI 兼容接口

1. 服务信息

1.1 端点

OlliteRT 运行在红米 K50 上，暴露 OpenAI 兼容 HTTP 接口：

```
Base URL:  http://192.168.1.14:8000/v1
协议:      HTTP/1.1, SSE (流式)
认证:      无 (未开启 Bearer Token)
```

1.2 可用端点

方法	路径	说明
POST	/v1/chat/completions	对话补全（OpenAI 格式，支持流式）
POST	/v1/completions	文本补全
POST	/v1/messages	Anthropic Claude 格式
GET	/v1/models	列出已加载模型
GET	/health	健康检查

1.3 前置条件

- K50 与后端服务在同一局域网（Wi-Fi）
- K50 上 OlliteRT 已启动并加载了模型
- App 设置 → 监听地址 = "局域网（所有接口）[0.0.0.0]"，不能是仅本机
- 电池优化设为"不受限制"，最近任务锁定 OlliteRT，防止 MIUI 杀后台

2. Maven 依赖

Spring AI 1.0.x（GA）。在 pom.xml 中加入：

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
</parent>

<dependencies>
    <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-starter-model-openai</artifactId>
        <version>1.0.0</version>
    </dependency>
</dependencies>

<repositories>
    <repository>
        <id>spring-milestones</id>
        <url>https://repo.spring.io/milestone</url>
    </repository>
</repositories>
```

3. application.yml 配置

```yaml
spring:
  ai:
    openai:
      # 指向手机上的 OlliteRT，结尾不要带 /v1
      base-url: http://192.168.1.14:8000
      # OlliteRT 不校验 Key，随便填一个非空字符串即可
      api-key: ollite-local
      chat:
        options:
          # 模型名通过 GET /v1/models 查，下面是示例
          model: gemma-3n-e2b
          temperature: 0.7
          # max-tokens 视模型而定，K50 上建议 1024~2048
          max-tokens: 1024
      # 关闭 Spring AI 默认连 OpenAI 官方的 embedding 自动配置
      embedding:
        enabled: false
```

注意：base-url 不要带 /v1，Spring AI 会自动拼接。如果开启了 Bearer Token，api-key 填手机上显示的那串。

4. 快速开始：ChatClient 调用

4.1 配置类

```
@Configuration
public class AiConfig {
    @Bean
    public ChatClient chatClient(ChatClient.Builder builder) {
        return builder.build();
    }
}
```

4.2 最简单的调用

```
@RestController
@RequestMapping("/ai")
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient chatClient) {
        this.chatClient = chatClient;
    }

    @GetMapping("/chat")
    public String chat(@RequestParam String q) {
        return chatClient.prompt()
                .user(q)
                .call()
                .content();
    }
}
```

4.3 带 System Prompt

```
String answer = chatClient.prompt()
    .system("你是一个二手交易平台的客服助手，回答要简洁友好。")
    .user("我想退货，怎么操作？")
    .call()
    .content();
```

5. 流式调用（SSE）

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> stream(@RequestParam String q) {
    return chatClient.prompt()
            .user(q)
            .stream()
            .content();
}
```

前端用 EventSource 或 fetch ReadableStream 接收，逐字渲染。OlliteRT 已支持逐 token 流式输出。

6. 多轮对话

```
// 方式一：手动维护历史
List<Message> history = new ArrayList<>();
history.add(new UserMessage("你好，我叫小明"));
history.add(new
AssistantMessage(chatClient.prompt().messages(history).call().content()));
history.add(new UserMessage("我叫什么名字？"));
String reply = chatClient.prompt().messages(history).call().content();

// 方式二：推荐使用 ChatMemory（按 sessionId 自动维护）
// 需要引入 spring-ai-starter-model-openai 的 chat memory 自动配置
String answer = chatClient.prompt()
    .user(q)
    .advisors(new MessageChatMemoryAdvisor(chatMemory, "session-001", 20))
    .call()
    .content();
```

7. Spring AI 与 HTTP 字段对应

Spring AI 配置/方法	实际发送到 OlliteRT 的字段
`spring.ai.openai.chat.options.model`	请求体 `model` 字段
`.system("...")`	`messages` 中 `role=system`
`.user("...")`	`messages` 中 `role=user`
`temperature / max-tokens / top-k / top-p`	请求体同名顶层字段
`.stream()`	请求体 `stream=true`，返回 SSE
`ChatResponse.getResult().getOutput().getText()`	`choices[0].message.content`

8. 常见问题排查

8.1 连接超时 / 连不上

- 确认 K50 和电脑在同一 Wi-Fi，关掉手机 VPN
- K50 上 App 设置 → 监听地址必须是 0.0.0.0，不能是 127.0.0.1
- K50 防火墙/MIUI 网络助手，确认 OlliteRT 允许 WLAN 联网
- 在电脑浏览器打开 http://192.168.1.14:8000/health，能返回 JSON 才算通

8.2 404 / model not found

- base-url 不要带 /v1，Spring AI 会自动拼
- model 字段必须和 GET /v1/models 返回的 id 完全一致，区分大小写
- App 里先点"启动服务器"加载模型，未加载时调接口会 404

8.3 回答很慢 / 首字等待久

- K50 是手机端 CPU/GPU 推理，首 token 通常 2~10 秒，属正常
- Spring / 网关的超时要调大：server.tomcat.connection-timeout、spring.mvc.async.request-timeout 设为 120000ms 以上
- 选小模型（Gemma 3n E2B）比大模型快很多

8.4 流式输出一次性返回

- App 设置 → 模型行为 → 工具 Schema 注入保持默认（开启）
- Spring AI 侧确认用 .stream() 而不是 .call()
- Nginx 反代时要关 proxy_buffering，否则会缓冲整个响应

8.5 应用被系统杀死后连不上

- MIUI 设置 → 应用 → OlliteRT → 省电策略 → 无限制
- 最近任务界面下拉 OlliteRT 卡片加锁
- App 设置 → 开机自启动 → 打开

9. 一行 curl 验证

```
curl http://192.168.1.14:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d "{\"model\":\"gemma-3n-e2b\",\"messages\":[{\"role\":\"user\",\"content\":\"
你好\"}]}"
```